



Lab Sheet-02

Data Structure I (BTCS209P)

B.Tech III Semester (Session 2026-2027)

Objective:

To understand and implement searching and sorting techniques using C++, and to analyze their working, algorithms, and time complexity.

Requirements:

C++ Compiler / IDE

VS Code, Code::Blocks, Dev-C++, or any suitable C++ environment

Basic knowledge of arrays, loops, functions, and conditional statements

Concepts Covered:

1. Searching

Searching is the process of finding a particular element (key) in a collection of data such as an array.

Linear Search

Binary Search

Linear Search checks elements one by one. Binary Search repeatedly divides a sorted array into two halves and eliminates the half that cannot contain the key.

2. Sorting

Sorting is the process of arranging data elements in a particular order, usually ascending or descending.

Bubble Sort

Insertion Sort

Selection Sort

Merge Sort

Quick Sort

3. Important Complexity Concepts

Best Case, Average Case, and Worst Case

Time Complexity

Space Complexity

Comparison-based sorting

Divide and Conquer



Searching Algorithms

1. Linear Search

Linear Search compares the key with each element from the beginning of the array until the key is found or the array ends.

Example:

Array = [10, 25, 7, 18, 30]

Key = 18

10 → 25 → 7 → 18

Time Complexity: Best Case – $O(1)$, Average Case – $O(n)$, Worst Case – $O(n)$

2. Binary Search

Binary Search works only on a sorted array. It compares the key with the middle element and repeatedly reduces the search range by half.

Example:

Array = [5, 10, 15, 20, 25, 30, 35]

Key = 25

Middle = 20

25 > 20 → Search right half

Middle = 30

25 < 30 → Search left half

25 found

Time Complexity: Best Case – $O(1)$, Average Case – $O(\log n)$, Worst Case – $O(\log n)$

Sorting Algorithms

1. Bubble Sort

Repeatedly compares adjacent elements and swaps them if they are in the wrong order.

Best: $O(n)$, Average: $O(n^2)$, Worst: $O(n^2)$, Space: $O(1)$

2. Insertion Sort

Builds the sorted array one element at a time by inserting each element into its correct position.

Best: $O(n)$, Average: $O(n^2)$, Worst: $O(n^2)$, Space: $O(1)$

3. Selection Sort

Repeatedly selects the minimum element from the unsorted part and places it at the beginning of that part.

Best: $O(n^2)$, Average: $O(n^2)$, Worst: $O(n^2)$, Space: $O(1)$

4. Merge Sort

Uses Divide and Conquer: divides the array into smaller parts, sorts them, and merges the sorted parts.

Best: $O(n \log n)$, Average: $O(n \log n)$, Worst: $O(n \log n)$, Space: $O(n)$



5. Quick Sort

Uses Divide and Conquer by selecting a pivot, partitioning the array around the pivot, and recursively sorting the two parts.

Best: $O(n \log n)$, Average: $O(n \log n)$, Worst: $O(n^2)$, Space: $O(\log n)$ average recursion stack

Lab Exercises

1. Write a C++ program to implement Linear Search on an array. Display the position of the key if it is found; otherwise display an appropriate message.
2. Write a C++ program to implement Binary Search on a sorted array. Display the position of the key if it is found.
3. Write a C++ program to compare Linear Search and Binary Search for a given array and display their results.
4. Write a C++ program to implement Bubble Sort in ascending order. Display the array after each pass.
5. Write a C++ program to implement Bubble Sort in descending order.
6. Write a C++ program to implement Insertion Sort in ascending order. Display the array after each insertion/pass.
7. Write a C++ program to implement Selection Sort in ascending order. Display the minimum element selected in each pass.
8. Write a C++ program to implement Merge Sort using separate functions for merging and sorting.
9. Write a C++ program to implement Quick Sort using the last element as the pivot.
10. Write a C++ program to implement Quick Sort and display the array after each partition.
11. Write a menu-driven C++ program that allows the user to choose Linear Search or Binary Search for a given array.
12. Write a menu-driven C++ program that allows the user to choose Bubble Sort, Insertion Sort, or Selection Sort.
13. Write a C++ program to sort an array using Merge Sort and then search for a given element using Binary Search.
14. Write a C++ program to implement any two sorting algorithms and compare their number of comparisons for the same input array.
15. Write a C++ program to implement any three sorting algorithms and display their Best, Average, and Worst-case time complexities as comments/output.



Practice / Analysis Questions

1. Why does Binary Search require a sorted array?
2. Why is the time complexity of Binary Search $O(\log n)$?
3. Why can Bubble Sort have $O(n)$ best-case complexity?
4. What is the difference between Insertion Sort and Selection Sort?
5. Why is Merge Sort considered a Divide and Conquer algorithm?
6. What is a pivot in Quick Sort?
7. Why can Quick Sort have $O(n^2)$ worst-case time complexity?
8. Which sorting algorithms in this lab are stable? Verify based on the implementation used.
9. Which sorting algorithm requires additional $O(n)$ space in its standard array implementation?
10. Compare Quick Sort and Merge Sort in terms of time complexity, space usage, stability, and in-place operation.

Instructions & Guidelines

Write a separate C++ program for each assigned exercise. Use functions wherever appropriate to make the program modular and easy to understand. Test every program with different inputs, including edge cases. For Binary Search, ensure that the input array is sorted before performing the search.

Display the input array and the final result clearly.

Add comments to important parts of the program.

Mention the time and space complexity of each implemented algorithm.

Do not use built-in sorting functions such as `sort()` for the implementation of the five sorting algorithms.

Submission

Submit the C++ source code files for the assigned tasks along with a brief report containing the algorithm, sample input/output, and time and space complexity analysis.

